Battery Health Monitor

Hardware setup and run guide – single 18650 Li-ion cell test rig

This guide takes you from a box of parts to a working system: wiring the test rig, setting up the laptop, uploading the ESP32 code, running a real discharge test, and recording the results for the report. Follow the parts in order. **Part A is done once.** Parts D and E are what you repeat for every cell, and on defence day.

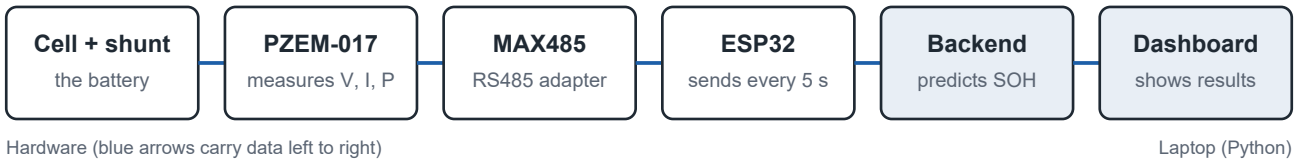


Figure 1: How the parts connect. The meter measures the cell; the ESP32 sends a reading every 5 seconds over WiFi; the laptop detects each discharge and predicts State of Health (SOH) and Remaining Useful Life (RUL).

How a test works, in one paragraph

You fully charge one cell, then discharge it at a steady 2.0 A until it reaches 2.7 V. While it discharges, readings stream to the laptop. The moment the load stops, the backend treats that as the end of one discharge cycle, computes four numbers from it (mean voltage, mean current, mean power and energy delivered), and the trained models predict the cell's health. You compare that prediction with the cell's **true** health, which the electronic load tells you (mAh delivered ÷ rated mAh). Doing this with a new cell and a worn cell shows the system can tell them apart.

> **Why a single 2 Ah cell and a steady 2 A?**
> The models learned from NASA's single 2 Ah 18650 cells discharged at a constant 2 A. The closer your test matches that, the more meaningful the prediction. A 12 V pack or a VRLA battery gives readings far outside what the models have seen, so its prediction means nothing. Test one cell at a time; buy 2–3 cells so you can compare a good one with a worn one.

Parts list

Item	What to get	Why
18650 Li-ion cells (2–3)	Rated about 2000 mAh. One new from a known brand; one or two worn cells from an old laptop battery pack. Not LiFePO4, not "9900 mAh" fakes.	Matches the 2 Ah cells the models were trained on.
18650 cell holder	Single-cell holder with thick leads.	Safe, solid contact.
Electronic load	Constant-current type (for example a DL24), set to 2.0 A with a 2.7 V cut-off. It must show mAh delivered.	Steady 2 A like the training data; its mAh reading gives the true SOH.
Li-ion charger	TP4056 module (4.2 V) or a proper 18650 charger.	Fully charges the cell before each test.
PZEM-017 + shunt	The 50 A shunt is best for a 2 A test; a 200 A shunt also works.	Measures voltage and current.
5 V USB adapter + cable	Any phone charger with a micro-USB cable.	Powers the PZEM-017 (a single cell is below the 7 V it needs).
MAX485 module	TTL-to-RS485 board.	Lets the ESP32 talk to the PZEM-017.

Item	What to get	Why
ESP32 Dev board	ESP32-WROOM dev module + USB cable.	Reads the meter and sends data over WiFi.
Resistors	One 1 kΩ and one 2 kΩ.	Protects the ESP32's 3.3 V input from the MAX485's 5 V output.
Jumper wires, multimeter	Male/female jumpers; any digital multimeter.	Wiring and accuracy checks.

Safety – read before touching the cell
• Never discharge below 2.5 V, and never short the cell's terminals (it can deliver very high current).
• Wire everything with the cell **out of the holder** and the load and charger **off**.
• Do not leave the cell charging or discharging unattended. Stop at once if it gets hot, swells or smells.
• Never connect GPIO27 or any ESP32 pin to the battery.

Part A – One-time laptop setup

A1. Python environment

Skip this if you have already run the project on this laptop. Open PowerShell in the project folder (`Battery-predictive-maintenance`) and run:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If PowerShell refuses to run the activate script, run this once, then try again:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

A2. Allow the ESP32 through the Windows Firewall

Open **PowerShell as Administrator** (right-click the Start button) and run once:

```
netsh advfirewall firewall add rule name="Battery backend 8000" `
    dir=in action=allow protocol=TCP localport=8000
```

Without this, the ESP32 joins WiFi but every reading fails to reach the laptop.

A3. Arduino IDE

1. Install the Arduino IDE from arduino.cc.
2. Add ESP32 support: **Tools › Board › Boards Manager**, search **esp32**, install **esp32 by Espressif Systems**.
3. Add the libraries: **Tools › Manage Libraries**, install **ModbusMaster** (by Doc Walker) and **ArduinoJson** (by Benoit Blanchon).
4. Open `firmware/esp32_firmware_v2/esp32_firmware_v2.ino` from the project folder.

A4. Edit four lines in the firmware

```
const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
const char* SERVER_URL    = "http://192.168.1.100:8000/readings";  // your laptop IP
#define SHUNT_RANGE_CODE 0x0002   // 0x0001 = 50 A, 0x0000 = 100 A, 0x0002 = 200 A, 0x0003 = 300 A
```

- **Laptop IP:** run `ipconfig` in PowerShell and copy the **IPv4 Address** (or double-click `start_system.bat`, which prints it). Put it in `SERVER_URL`, keeping `:8000/readings`.
- **WiFi:** the ESP32 only works on **2.4 GHz**. A phone hotspot is fine – turn on "maximise compatibility". The laptop and the ESP32 must be on the **same** network.
- **Shunt:** set `SHUNT_RANGE_CODE` to match the shunt you fitted. A wrong setting makes current read half or double the true value.
- **Changing networks** (for example on defence day) changes the laptop IP: update SERVER_URL and upload again.

A5. Upload to the ESP32

1. Plug the ESP32 into the laptop by USB.
2. **Tools › Board › esp32 › ESP32 Dev Module**, then **Tools › Port** › the new COM port.
3. Click **Upload** (the right-arrow button). If it hangs at "Connecting...", hold the ESP32's **BOOT** button until uploading starts.

Part B – Wiring

Everything unplugged while you wire: cell out of the holder, load and charger off, ESP32 and USB adapter disconnected.

B1. Current path and measurement wires

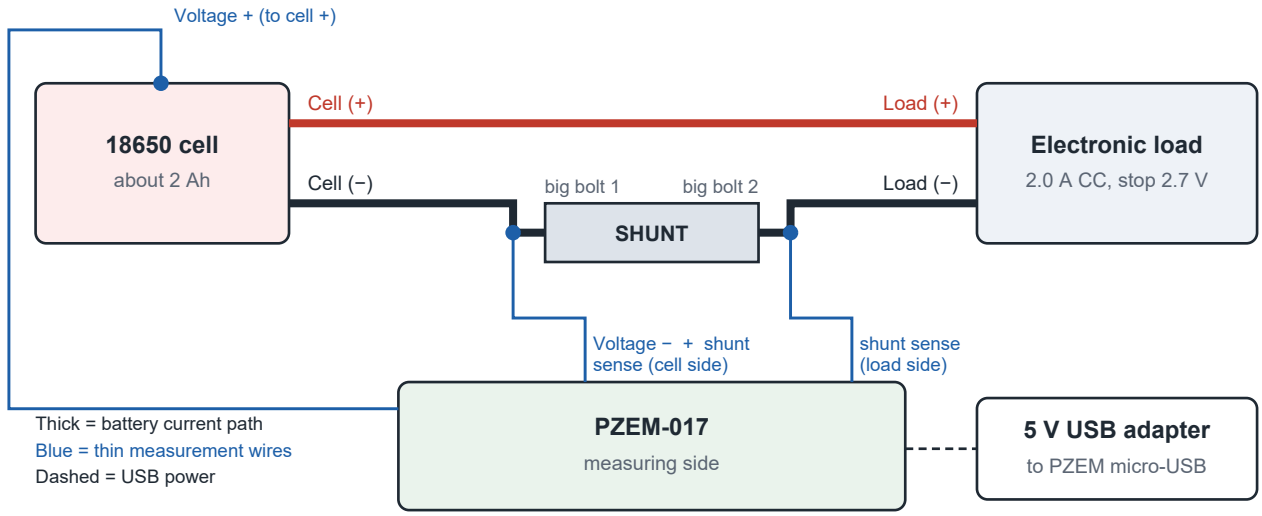


Figure 2: The shunt sits in the **negative** *lead between the cell and the load. The PZEM-017 measures the cell voltage directly and the tiny voltage across the shunt.*

1. **Cell (+)** to **load (+)** with a thick wire.
2. **Cell (−)** to shunt **big bolt 1**; shunt **big bolt 2** to **load (−)**.
3. PZEM-017 **voltage wires** across the cell: voltage + to cell (+), voltage − to cell (−).
4. PZEM-017 **shunt sense wires** (thin) to the two **small screws** on the shunt.
5. Plug the **5 V USB adapter** into the PZEM-017's **micro-USB** port.

> **Terminal names differ between PZEM-017 batches**
> Follow the wiring diagram printed on your PZEM-017 or in its manual for the exact order of its four measuring terminals. What matters is: the shunt is in the negative lead, the voltage wires go across the cell, and the sense wires go to the shunt's small screws. If the current later reads 0.00 A while the load is running, swap the two sense wires.

B2. Data wiring: PZEM-017 → MAX485 → ESP32

* RO goes to GPIO16 through a 1 kΩ / 2 kΩ divider (see B4).

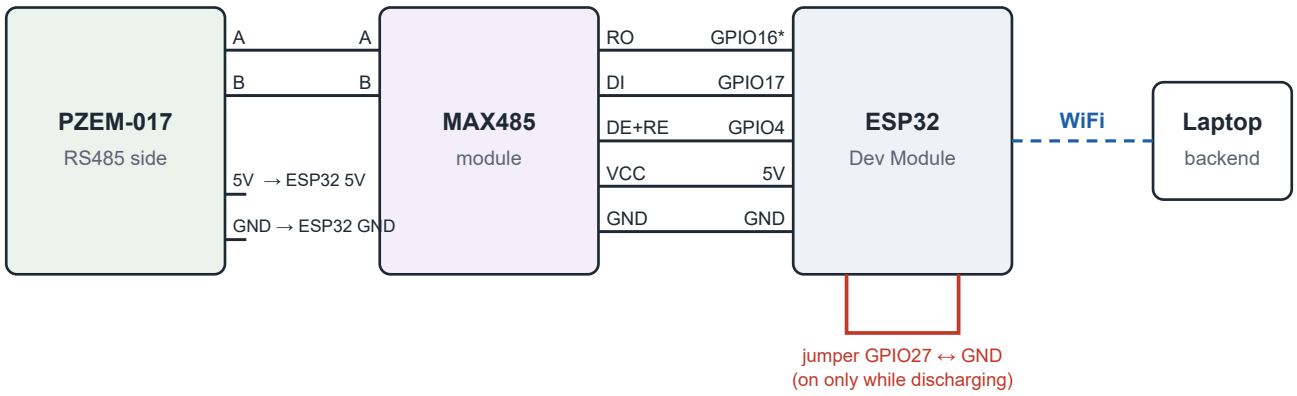


Figure 3: RS485 link and ESP32 connections.

From	To	Note
PZEM-017 RS485 A	MAX485 A	
PZEM-017 RS485 B	MAX485 B	If the meter never answers, swap A and B.
PZEM-017 RS485 5V	ESP32 5V (VIN)	**Required.** The RS485 side of the PZEM-017 needs its own 5 V.
PZEM-017 RS485 GND	ESP32 GND	Common ground.
MAX485 VCC	ESP32 5V (VIN)	
MAX485 GND	ESP32 GND	Common ground.
MAX485 RO	ESP32 GPIO16 (RX2)	Through the divider in B4.
MAX485 DI	ESP32 GPIO17 (TX2)	
MAX485 DE and RE (joined)	ESP32 GPIO4	Direction control: HIGH send, LOW receive.

B3. Discharge signal (GPIO27)

The PZEM-017 reports current without a sign, so the ESP32 needs to be told when the cell is discharging. Connect a jumper wire from **GPIO27 to GND** for the whole discharge, and remove it while charging. (If your load switch has a spare contact, wire that contact between GPIO27 and GND instead.) **Never connect GPIO27 to the battery.**

B4. Voltage divider on RO

The MAX485 outputs 5 V but the ESP32 input takes 3.3 V. Build this divider:

```
MAX485 RO ---[ 1 kΩ ]---+--- ESP32 GPIO16
                        |
                    [ 2 kΩ ]
                        |
                       GND
```

5 V × 2 kΩ ÷ 3 kΩ = 3.3 V at GPIO16.

B5. Checks before power

- Multimeter across the cell: between 2.7 V and 4.2 V.
- No bare wires touching; the shunt bolts are tight.
- ESP32 GND, MAX485 GND and PZEM-017 RS485 GND are all joined.

- Nothing from the ESP32 touches the battery wires.

Part C – Start the software

1. In the project folder, double-click `start_system.bat`. Two windows open:
   • **Backend** – wait for `All ML models loaded successfully.`
   • **Dashboard** – your browser opens the dashboard page.
   The first window also prints your laptop IP. Check it matches the firmware's SERVER_URL.
2. Plug the ESP32 into the laptop by USB (this also powers it). In the Arduino IDE open **Tools › Serial Monitor** and set it to **115200** baud.

You should see:

```
[WiFi] Connected, IP 192.168.x.x
[PZEM] current range already set (code 2)
[PZEM] V=4.15 V  I=0.00 A  P=0.0 W  E=0 Wh
```

and the dashboard's **Voltage** card updates every 5 seconds. If not, see Troubleshooting at the end.

Manual start (instead of start_system.bat)

Open two PowerShell windows in the project folder:

```
# window 1 - backend
.\venv\Scripts\Activate.ps1
cd backend
python main.py

# window 2 - dashboard
.\venv\Scripts\Activate.ps1
cd dashboard
streamlit run dashboard.py
```

C2. Check the meter against the multimeter

Before the first real test, confirm the readings are accurate. With the load running at 2.0 A for a minute (jumper on), compare:

- Cell voltage: multimeter across the cell terminals vs the dashboard Voltage card.
- Current: the electronic load's display (or a multimeter in series) vs the dashboard Current card (shown as about −2.00 A; the minus sign means discharging).
- Both should agree within about 1–2%. Record the pairs in the accuracy table in Part E.

Then stop the load, remove the jumper, and recharge the cell fully before the real test.

Part D – Run one test cycle

1. **Charge.** Load disconnected, jumper **off**. Connect the charger and charge until it shows full (4.2 V). Disconnect the charger and let the cell rest for 30 minutes.
2. **Set the load** to constant current **2.0 A** with a cut-off of **2.7 V**, and reset its mAh counter to zero.
3. **Jumper on** (GPIO27 to GND), then **start the load**. The backend window prints `Discharge started`.
4. **Wait.** The load stops by itself at 2.7 V – about an hour for a healthy cell, less for a worn one. Keep the laptop awake (turn off sleep) and the backend and dashboard windows open.
5. **Prediction.** When the load stops, the current drops to zero and the backend closes the cycle. It prints `Cycle 1 complete -- SOH: ...% | RUL: ... days | Alert: ...` and the dashboard's gauge, RUL panel and alert banner update. Then take the jumper **off**.
6. **Record** the results in Part E: the mAh from the load, the predicted SOH, RUL and alert, and a screenshot of the dashboard.

7. **Repeat** steps 1–6 with each worn cell. Photograph the rig for Figure 4.1 of the report.

Working out the true SOH
True SOH (%) = mAh delivered by the load ÷ 2000 × 100 (use 2000 mAh, the rated capacity the models assume). Example: a worn cell delivers 1480 mAh → true SOH = 1480 ÷ 2000 × 100 = 74%. Compare this with the predicted SOH on the dashboard.

Part E – Results sheet

Fill this in during testing. It becomes the test-rig table in Chapter 4 of the report.

Measurement accuracy (from C2)

Measurement	PZEM-017 (dashboard)	Reference (multimeter / load)	Difference (%)
Open-circuit voltage before discharge (V)			
Voltage under 2 A load (V)			
Discharge current (A)			

Prediction results (one row per cell)

Cell	Source / age	Discharge time (min)	mAh delivered	True SOH (%)	Predicted SOH (%)	Predicted RUL (days)	Alert
1	New						
2	Worn (laptop pack)						
3							

What a good result looks like: the new cell shows a high SOH and a NORMAL alert, the worn cell a clearly lower SOH, and each prediction lands within a few percentage points of its true SOH.

Part F – Demo without hardware (backup plan)

If the rig is not available, the simulator replays a real NASA discharge cycle into the backend exactly as the ESP32 would. With `start_system.bat` running, open another PowerShell in the project folder:

```
.\venv\Scripts\Activate.ps1
python tools\simulate_discharge.py --stage early   # healthy cell -> green gauge, NORMAL
python tools\simulate_discharge.py --stage late    # worn cell    -> red gauge, URGENT
```

Each run takes about 20 seconds. To clear old results, close both windows, delete `backend\battery.db`, and start again.

Troubleshooting

What you see	What to do
[PZEM] read error / cannot read current range	Swap RS485 A and B. Check the PZEM-017 RS485 5V and GND wires to the ESP32, and the USB adapter on the PZEM micro-USB. Check the RO divider.
WiFi never connects	Use a 2.4 GHz network; check the name and password (they are case-sensitive).
[HTTP] errors, or the dashboard never updates	Wrong laptop IP in SERVER_URL, backend not running, firewall rule (A2) missing, or the laptop and ESP32 are on different networks.
Current reads about half or double the true value	SHUNT_RANGE_CODE does not match the shunt fitted.
Current reads 0.00 A while the load runs	Shunt sense wires loose or swapped; check the small screws on the shunt.

What you see	What to do
Readings arrive but no prediction appears	The jumper was not on during the discharge (current must arrive as negative), or the load has not stopped yet.
Dashboard banner says models not loaded	ml/models/ is missing its .pkl files: run cd ml; python train_models.py (about 5 minutes).
Worked yesterday, not today	The laptop IP changed: re-check ipconfig, update SERVER_URL, upload again.
Dashboard shows "Backend offline"	The backend window was closed or crashed: run start_system.bat again.

Quick reference

Task	Command (PowerShell, in the project folder)
Start everything	Double-click start_system.bat
Activate the Python environment	.\venv\Scripts\Activate.ps1
Backend only	cd backend; python main.py
Dashboard only	cd dashboard; streamlit run dashboard.py
Simulated test	python tools\simulate_discharge.py --stage late
Retrain the models	cd ml; python train_models.py
Check model accuracy	cd ml; python eval_saved_models.py
Regenerate report tables and figures	python tools\report_results.py
Find the laptop IP	ipconfig (IPv4 Address)
Clear all stored results	Close both windows, delete backend\battery.db